

WebdriverIO MCP

Automatización con MCP, WebdriverIO y Appium

Guía completa en español: qué es el servidor MCP de WebdriverIO, cómo levantar Appium paso a paso y cómo interactuar con todo el stack desde opencode.

Contenido traducido y resumido de webdriver.io/docs/mcp · Generado con opencode

Contenido

#	Sección	Resumen
01	¿Qué es el servidor MCP?	Qué es @wdio/mcp, sus ventajas y su interfaz unificada
02	¿Qué puede hacer?	Capacidades de navegador y de apps móviles
03	Arquitectura del stack	IA ↔ MCP ↔ WebdriverIO ↔ Appium ↔ dispositivo
04	Instalación en opencode	Config global con el bloque mcp y variables de Appium
05	Las 25 herramientas	Tools de navegador y de móvil, y manejo automático
06	Selectores	CSS, XPath, UiAutomator, predicados iOS y más
07	Levantar Appium paso a paso	Instalación, drivers y arranque en Android e iOS
08	Interactuar con el stack	Ejemplos de prompts y flujo completo con tu APK
09	Solución de problemas	Fallos típicos y qué revisar en cada plataforma
10	Recursos oficiales	Documentación, repositorio y paquetes npm

En este proyecto ya tenés instalado Appium 2.19.0, el driver uiautomator2 y un emulador Android en marcha (`emulator-5554`). La suite E2E del proyecto usa WebdriverIO + **Mocha** (`features/specs/*.spec.ts`); esta guía es solo para la automatización interactiva vía MCP. Detalles de la suite en el `README.md` .

01 · ¿Qué es el MCP de WebdriverIO?

Es un servidor **Model Context Protocol (MCP)** (`@wdio/mcp`) que permite a asistentes de IA (opencode, Claude Desktop, Claude Code) automatizar navegadores web y aplicaciones móviles nativas.

Mobile-First

- A diferencia de MCP servers solo para navegador, soporta iOS y Android nativos mediante Appium.
- Apps híbridas: cambio de contexto nativo ↔ WebView.

Selectores multiplataforma

- Detección inteligente de elementos: genera varias estrategias a la vez (Accessibility ID, XPath, UiAutomator, predicados iOS).

Ecosistema WebdriverIO

- Construido sobre el probadísimo framework WebdriverIO y su ecosistema de servicios y reporters.

Interfaz unificada

- 🖥️ Navegadores de escritorio (Chrome con o sin headless).
- 📱 Apps móviles nativas (simulador iOS / emulador Android / dispositivos reales vía Appium).
- 🏠 Apps híbridas (nativo + WebView).

02 · ¿Qué puede hacer?

Navegador (Chrome)

- Lanzar Chrome en modo con ventana o headless, con dimensiones personalizadas y navegación inicial.
- Navegar, hacer clic, escribir, hacer scroll.
- Analizar la página: árbol de accesibilidad y elementos visibles (con paginación y filtros).
- Capturas de pantalla optimizadas (redimensionadas y comprimidas a máx. 1 MB).
- Gestión de cookies (obtener, crear, borrar).
- Ejecutar JavaScript en el contexto del navegador.

Apps móviles (iOS/Android)

- Ciclo de vida de sesión y de la app (estado, activar/terminar).
- Gestos táctiles: tap, swipe, drag & drop.
- Detección de elementos con múltiples estrategias y paginación.
- Cambio de contexto en apps híbridas.
- Control del dispositivo: rotación, teclado, geolocalización.
- Permisos y alertas automáticas.
- Comandos móviles de Appium (pressKey, deepLink, shell...).

Ojo con el contexto: usar un servidor MCP suma tokens al contexto. Conviene ser selectivo con los MCPs habilitados; servidores grandes (p. ej. el de GitHub) pueden consumir mucho.

03 · Arquitectura: el stack completo

El servidor MCP actúa de puente entre el asistente de IA y los navegadores o dispositivos móviles:

```
Asistente IA → @wdio/mcp → WebDriverIO → Appium → Dispositivo
    ↑           ↑           ↑           ↑           ↑
  opencode  Servidor MCP  Cliente  Puerto  Chrome · iOS ·
    · stdio      · WebDriver  4723    Android
```

Gestión de sesiones

- Modelo de sesión única: solo un navegador o app activa a la vez.
- El estado de la sesión se mantiene globalmente entre llamadas.
- Auto-detach: las sesiones con estado preservado (`noReset: true`) se desvinculan solas al cerrar.

Detección de elementos

- Web: script optimizado que encuentra los elementos visibles e interactivables (selectores CSS, IDs, clases, ARIA). Filtra por viewport por defecto.
- Móvil: parseo XML eficiente de la página (2 llamadas HTTP frente a 600+), con clasificación por plataforma.

Estrategias de localización generadas por elemento (móvil)

Estrategia	Descripción
Accessibility ID	Multiplataforma, la más estable (el recomendado por WebdriverIO)
Resource ID / Name	Atributo nativo de Android / iOS
Texto / Label	Coincidencia por texto visible o etiqueta
XPath	XPath completo y simplificado
UiAutomator / Predicates	Selectores nativos de Android (UiAutomator) e iOS (Predicates)

Recomendación oficial: usa siempre que puedas Accessibility ID (`~loginButton`): es la estrategia multiplataforma más estable y se usa tanto en iOS como en Android.

04 · Instalación y configuración en opencode

La forma más sencilla es usar `npx` sin instalación local:

```
npx @wdio/mcp
```

Para que opencode lo cargue en todos tus proyectos, agrégalo al archivo de configuración global `~/.config/opencode/opencode.jsonc` :

```
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "wdio-mcp": {
      "type": "local",
      "command": ["npx", "-y", "@wdio/mcp"],
      "enabled": true
    }
  }
}
```

Importante: tras guardar la configuración hay que cerrar y reabrir opencode. Pon `enabled: false` para desactivarlo temporalmente sin borrarlo.

Verifica que el servidor quedó conectado:

```
opencode mcp list
```

Variables de entorno de Appium

El servidor MCP configura su conexión a Appium con estas variables:

Variable	Descripción	Default
APPIUM_URL	Host de Appium	127.0.0.1

APPIUM_URL_PORT	Puerto	4723
APPIUM_PATH	Ruta base	/

Optimización y robustez: el servidor usa formato TOON (notación orientada a tokens) para comunicaciones ligeras, comprime las capturas a máx. 1 MB con Sharp, filtra por viewport y devuelve los errores como texto sin perder el estado de la sesión.

TypeScript: el paquete está escrito en TS con tipos completos, así que también puede integrarse y extenderse programáticamente con autocompletado y seguridad de tipos.

05 · Las 25 herramientas disponibles

Navegador (Chrome)

Herramienta	Descripción
start_browser	Lanza Chrome (con URL inicial opcional)
close_session	Cierra o desvincula la sesión
navigate	Navega a una URL
click_element	Hace clic en un elemento
set_value	Escribe texto en un campo
get_visible_elements	Elementos visibles/interactuables (con paginación)
get_accessibility	Árbol de accesibilidad (con filtros)
take_screenshot	Captura optimizada automáticamente
scroll	Hace scroll arriba/abajo
get_cookies · set_cookie · delete_cookies	Gestión de cookies
execute_script	Ejecuta JavaScript en el navegador

Móvil (iOS / Android)

Herramienta	Descripción
start_app_session	Lanza una app en iOS/Android
tap_element	Toca un elemento o unas coordenadas
swipe	Desliza en una dirección
drag_and_drop	Arrastra y suelta entre ubicaciones
get_app_state	Comprueba si la app está en ejecución
get_contexts · switch_context	Cambio de contexto en apps híbridas (nativo ↔ WebView)
rotate_device	Rota a vertical/horizontal

get_geolocation · set_geolocation	Obtiene o fija las coordenadas GPS
hide_keyboard	Oculto el teclado en pantalla
execute_script	Comandos móviles de Appium (pressKey, deepLink, shell...)

06 · Sintaxis de selectores

El servidor soporta múltiples estrategias de localización, tanto en web como en móvil.

Web · CSS / XPath / Texto

CSS

```
button.my-class
#element-id
[data-testid="login"]
```

XPath

```
//button[@class='submit']
//a[contains(text(), 'Click')]
```

Texto (WebdriverIO)

```
button=Exact Button Text
a*=Partial Link Text
```

Recomendado: en web, usa `data-testid` cuando exista; en móvil, prioriza el Accessibility ID.

Móvil · estrategias multiplataforma

Accessibility ID (recomendado) — funciona en iOS y Android:

```
~loginButton
```

Android UiAutomator

```
android=new UiSelector()
.text("Login")
```

iOS Predicate String

```
-ios predicate string:label = "Login"
```

iOS Class Chain

```
-ios class chain:**/XCUIElementTypeButton[`label = "Login"`]
```

XPath (ambas plataformas)

```
//android.widget.Button[@text="Login"]
//XCUIElementTypeButton[@label="Login"]
```

«~» = **Accessibility ID**: la estrategia más estable y la que genera el servidor por defecto junto con las demás.

07 · Paso a paso: levantar Appium

Appium es el servidor que conecta al dispositivo con WebdriverIO. Requisitos y pasos exactos, separados por plataforma.

Requisitos previos

Plataforma	Requisitos
Navegador	Chrome instalado. WebdriverIO gestiona el chromedriver automáticamente
Base	Node.js $\geq$ 18 (recomendado: 22 LTS; @wdio/mcp usa dependencias que piden Node $\geq$ 22)
Android	Android Studio + Android SDK · ANDROID_HOME configurado · Emulador o dispositivo con adb
iOS	Xcode (desde la Mac App Store) · Command Line Tools · Simuladores o dispositivo real

Instalación y arranque · Android

1. Instalar Appium de forma global

```
npm install -g appium
```

2. Instalar el driver de Android (UiAutomator2)

```
appium driver install uiautomator2
```

3. Configurar variables de entorno (en tu shell)

```
export ANDROID_HOME=$HOME/Library/Android/sdk
export PATH=$PATH:$ANDROID_HOME/emulator:$ANDROID_HOME/platform-tools
```

4. Lanzar el emulador (o conectar un dispositivo real)

Android Studio → Device Manager. Verifica con `adb devices` : debe aparecer algo como `emulator-5554 device` .

5. Arrancar el servidor Appium

```
appium # escucha en 127.0.0.1:4723
```

Instalación y arranque · iOS

1. Instalar Xcode y las Command Line Tools

Xcode desde la App Store y luego:

```
xcode-select --install
```

2. Instalar Appium y el driver XCUITest

```
npm install -g appium
appium driver install xcuitest
```

3. Crear/ver simuladores y arrancar Appium

Xcode → Window → Devices and Simulators. Verifica con `xcrun simctl list devices`. Para dispositivos reales necesitas el UDID (40 caracteres). Luego ejecuta `appium`.

Verificación rápida de tu instalación

```
appium --version           # → 2.19.0 ✓
appium driver list --installed # → uiautomator2 ✓
adb devices                # → emulator-5554 device ✓
```

El flujo completo: `adb devices` (dispositivo) → `appium` (servidor) → `opencode` (MCP) → automatizar.

Ejemplo de estado listo para automatizar

Emulador encendido + appium en una terminal + opencode con el MCP conectado = puedes pedirle al asistente que controle la app.

08 · Cómo interactuar con todo el stack

Con el MCP activo, el asistente usa las tools automáticamente: solo tienes que pedirle lo que quieres lograr en lenguaje natural.

Ejemplos · Navegador

- "Abre Chrome y navega a <https://webdriver.io>"
- "Haz clic en el botón 'Get Started'"
- "Toma un screenshot de la página"
- "Encuentra todos los enlaces visibles"

Ejemplos · Móvil

- "Inicia mi app iOS en el simulador iPhone 15"
- "Toca el botón de login"
- "Desliza hacia arriba para hacer scroll"
- "Toma un screenshot de la pantalla actual"

Flujo completo de ejemplo (Android · con tu proyecto)

1. Enciende el emulador y Appium

Emulador arrancado (`adb devices` → `emulator-5554 device`) y en otra terminal: `appium`.

2. Pide iniciar la sesión con tu APK

En la raíz de webDriverIOMobile tienes `DriverIO.apk` . Pídele: “Inicia la app `DriverIO.apk` en el emulador Android”. El MCP llama a `start_app_session` con la ruta de la APK.

3. Interactúa

- o “Toca el botón de login” → `tap_element`
- o “Escribe 'demo' en el campo de usuario” → `set_value`
- o “Haz swipe hacia arriba” → `swipe`

4. Analiza y captura evidencia

- o “Toma un screenshot” → `take_screenshot` (comprimido a máx. 1 MB)
- o “Muéstrame el árbol de accesibilidad” → `get_accessibility`

Trucos y buenas prácticas

Práctica	Detalle
Sesión única	Solo puede haber un navegador o app activa. Cierra la sesión (<code>close_session</code>) antes de cambiar de blanco
Prioriza el Accessibility ID	Selectores <code>~id</code> que no dependen del layout son los más robustos multiplataforma
Contextos híbridos	En apps nativo+WebView: <code>get_contexts</code> lista los contextos y <code>switch_context</code> cambia entre nativo y webview
Primera ejecución	La descarga de <code>npx @wdio/mcp</code> puede superar los 30 s de timeout. Precalienta con <code>npx -y @wdio/mcp</code> una vez

Los errores nunca rompen la sesión: el servidor MCP devuelve los errores como contenido de texto y preserva el estado de la sesión, así que puedes encadenar intentos sin reiniciar.

09 · Solución de problemas

Síntoma	Qué revisar
El navegador no arranca	Chrome instalado · puerto de depuración 9222 libre · probar modo headless si hay problemas de pantalla
Conexión Appium fallida	Servidor appium corriendo · <code>APPIUM_URL</code> y puerto correctos · driver instalado (<code>appium driver list</code>)
Problemas con simulador iOS	Xcode actualizado · simuladores disponibles (<code>xcrun simctl list devices</code>) · UDID correcto en dispositivos reales
Problemas con emulador Android	<code>ANDROID_HOME</code> configurado · emulador en marcha (<code>adb devices</code>)
MCP “failed / timeout”	Primera descarga de <code>npx @wdio/mcp</code> : precargar la caché y reintentar · Node ≥ 22 recomendado

Flujo de chequeo de tu instalación actual:

```
appium --version          # → 2.19.0 ✓  
appium driver list --installed # → uiautomator2 ✓  
adb devices              # → emulator-5554 device ✓
```

10 · Recursos oficiales

Documentación

- MCP WebdriverIO: <https://webdriver.io/docs/mcp>
- Configuración: <https://webdriver.io/docs/mcp/configuration>
- Transport y cloud: <https://webdriver.io/docs/mcp/transport>
- Especificación MCP: <https://modelcontextprotocol.io>

Código y paquetes

- Paquete npm: `@wdio/mcp`
- Repo GitHub: <https://github.com/webdriverio/mcp>
- Herramientas y selectores: <https://webdriver.io/docs/mcp/tools> · <https://webdriver.io/docs/mcp/selectors>

Créditos y atribución: contenido traducido y resumido de la documentación oficial de WebdriverIO MCP (webdriver.io/docs/mcp, © OpenJS Foundation y colaboradores de WebdriverIO 2026). Iconos: Twemoji (© 2020 Twitter, Inc.), licencia CC-BY 4.0 · ilustraciones y diseño generados con opencode.